faultbox: Cost and Containment of In-Process Fault Isolation with Memory Protection Keys

Meghashyam Sai Dontha, Blekinge Institute of Technology (BTH)

November 2026

Code and data: https://github.com/saishyamdontha/faultbox

Abstract

Systems that run untrusted code inside a host process, such as plugin hosts and language runtimes, depend on components that sit outside any sandbox. A memory-safety fault in one of them can crash or silently corrupt the whole process. This report compares four ways for a C host to call untrusted plugins: no isolation, process isolation (`fork` + `seccomp` strict), and two variants of in-process isolation using x86 memory protection keys (MPK/PKU) with signal-based recovery. Under injected faults, both isolation approaches keep host state intact in all cases, while the unisolated host crashes or is silently corrupted by 2 of 3 faulty plugins. The MPK domain switch adds about 21 ns per call, about 4,500x less than process isolation (109 µs). A naive MPK implementation costs 313 ns per call, and about 92% of that is signal-mask bookkeeping, not the switch itself. Finally, a plugin that executes `WRPKRU` bypasses MPK isolation entirely. A load-time scanner blocks this direct form of the attack, but not indirect forms, which shows the limits of MPK as a security boundary.

1. Motivation

Process isolation is the standard way to contain untrusted code, but every call then crosses a process boundary. In-process isolation keeps calls cheap by dividing a single address space into memory domains. MPK makes this possible in hardware: each page carries a 4-bit key, and a per-thread register (PKRU) controls access to each key, changed with the unprivileged `WRPKRU` instruction.

This project asks three practical questions:

1. Which faults does each isolation strategy actually contain?
2. What does each strategy cost per call and per recovery?
3. Where does MPK-based isolation stop working as a boundary?

2. Threat and fault model

The host owns one page of state (`host_state`) that must stay intact. Plugins are shared objects loaded with `dlopen` that export `plugin_run(int)`. Four plugins are tested:

Plugin	Behaviour	Category
`good`	Returns `2 * input`	control
`nullderef`	Dereferences a null pointer	crash fault
`oob_write`	Writes 8 bytes into `host_state`	silent corruption fault
`wrpkru_attack`	Executes `WRPKRU` with PKRU = 0, then writes `host_state`	malicious bypass

The primary model is a **buggy** plugin: accidental memory-safety errors. The malicious plugin is included to test where the MPK boundary fails, not to claim protection against hostile code.

3. Design

Mode	Isolation	Recovery
`none`	None; plugin shares the host address space	None
`fork`	Child process per call, `seccomp` strict mode; result via pipe	Child exits abnormally; host continues
`mpk`	`host_state` tagged with a key; access disabled while the plugin runs	SIGSEGV handler `siglongjmp`s to the host, restores PKRU, discards (`dlclose`) and reloads (`dlopen`) the plugin
`mpk_fast`	As `mpk`, with `sigsetjmp(buf, 0)` and an `SA_NODEFER` handler	As `mpk`
`mpk_safe`	As `mpk_fast`, plus a load-time scan rejecting files containing `0F 01 EF` (`WRPKRU`)	As `mpk`; flagged plugins are never loaded

Three implementation details turned out to matter:

- **PKRU after recovery.** The kernel resets PKRU when it delivers a signal, and `siglongjmp` skips `sigreturn`, so the host must restore access to its own key explicitly.
- **Signal mask.** `sigsetjmp(buf, 1)` saves the signal mask, which costs a syscall on every call. Skipping the save is only safe if the handler is installed with `SA_NODEFER`. Otherwise SIGSEGV stays blocked after the jump, and the next fault kills the process.
- **Strict seccomp exit.** glibc's `_exit()` uses `exit_group`, which strict mode forbids, so the child exits with `syscall(SYS_exit)`. Children are marked non-dumpable so faults do not produce core dumps that would distort timing.

4. Evaluation setup

- **Hardware/OS:** AMD Ryzen 9 8940HX (PKU supported), Ubuntu 24.04, gcc. Host built with `-O2`, plugins with `-O0` so the fault code is kept as written.
- **Containment:** each (mode, plugin) pair runs in a fresh host process; the outcome is classified from the exit status and a check of `host_state`.
- **Per-call cost:** 1,000,000 calls to `good` (10,000 for `fork`) after 1,000 warm-up calls, timed with `CLOCK_MONOTONIC`.
- **Fault + recovery:** mean over 1,000 calls to `nullderef`, including plugin reload for MPK modes.
- **Repetitions:** 5 runs per mode, pinned to one core with `taskset`; means reported.

5. Results

5.1 Containment

Mode	good	nullderef	oob_write	wrpkru_attack
none	ok	host crashed	host corrupted	host corrupted
fork	ok	contained	ok (child's copy)	ok (child's copy)
mpk	ok	contained, reloaded	contained, reloaded	**host corrupted**
mpk_fast	ok	contained, reloaded	contained, reloaded	**host corrupted**
mpk_safe	ok	contained, reloaded	contained, reloaded	rejected at load

In the MPK modes, the two faults are caught for different reasons. The out-of-bounds write raises `SEGV_PKUERR`: **the protection key is what stops the corruption**. The null dereference raises `SEGV_MAPERR`: MPK plays no part, and **the signal-based recovery is what keeps the host alive**. Process isolation contains the corrupting write without any fault, because the write lands in the child's copy-on-write memory.

5.2 Performance

Mode	Per call	Overhead vs `none`	Fault + recovery
none	2.9 ns	–	–
mpk_fast	23.9 ns	+21 ns	29.8 µs
mpk	313 ns	+310 ns	30.9 µs
fork	109,070 ns	+109 µs	110 µs

`mpk_safe` has the same call path as `mpk_fast`; its scan runs once at load time and was not benchmarked separately.

6. Discussion

The domain switch is cheap; bookkeeping around it is not. Removing the signal-mask save cuts MPK overhead from 313 ns to 24 ns, about 92%. For in-process isolation, the cost of the recovery machinery around each call can exceed the cost of the isolation itself by an order of magnitude, so it needs to be designed and measured as carefully as the boundary.

Recovery cost is dominated by reloading, not by fault handling. Both MPK modes recover in about 30 µs, despite a 13x difference in per-call cost. This suggests `dlclose`/`dlopen` dominates. This is an inference and has not yet been measured separately. Recovering by restoring a saved domain snapshot instead of reloading code could lower it.

MPK is a fault boundary first and a security boundary only with extra work. Because `WRPKRU` is unprivileged, any code running in the plugin domain can unlock every key. The scanner blocks the direct case, but a plugin can still call glibc's `pkey_set()`, where the instruction lives in libc rather than the plugin file.

Implication for runtimes that host untrusted code. Language runtimes such as WebAssembly engines already sandbox guest code, but host functions, system-interface implementations and third-party libraries run outside that sandbox. The results here suggest in-process domains are cheap enough to place around such components on every call. They also suggest that recovery design and resistance to `WRPKRU` misuse, not the switch cost, are the hard parts.

7. Limitations and future work

- Only one page is protected; the rest of host memory remains accessible to plugins in MPK modes.
- MPK modes do not restrict syscalls; only `fork` uses seccomp.
- The `WRPKRU` scanner misses library calls (`pkey_set`), `XRSTOR`, and code generated at runtime, and may produce false positives on arbitrary byte matches.
- Single-threaded: PKRU is per-thread and recovery state is global.
- No hang detection: a looping plugin blocks the host in MPK modes.
- Reloading resets plugin memory only, not external side effects.
- Results come from a single machine.

Future work: separate fault-handling and reload time; reject plugins that import `pkey_*` symbols; per-plugin keys so co-located plugins are isolated from each other; a watchdog for hangs; evaluation on Intel CPUs.

8. Conclusion

On this machine, MPK-based in-process isolation contained every accidental fault tested at about 21 ns per call, roughly 4,500x cheaper than process isolation. Most of the cost of a naive version came from signal-mask handling rather than the protection-key switch. MPK did not withstand a plugin that deliberately executes `WRPKRU`, and a simple load-time scan closes only part of that gap.

References

1. A. Vahldiek-Oberwagner et al., "ERIM: Secure, Efficient In-process Isolation with Protection Keys (MPK)," USENIX Security Symposium, 2019.
2. M. Hedayati et al., "Hodor: Intra-Process Isolation for High-Throughput Data Plane Libraries," USENIX Annual Technical Conference, 2019.
3. M. Gülmez et al., "Rewind & Discard: Improving Software Resilience using Isolated Domains," IEEE/IFIP DSN, 2023.
4. Linux man-pages: `pkeys(7)`, `pkey_alloc(2)`, `seccomp(2)`, `sigaction(2)`, `sigsetjmp(3)`.